

Project Proposal — Forecasting One Piece TCG Swiss Matches

Kiana Kiser · MLOps HS26 · I.BA_MLOPS · Repository (public): github.com/kianakiser/optcg-match-forecast

1 Problem statement

What and for whom. For every Swiss match at an online One Piece TCG tournament, predict the probability the seat-1 player wins, from both registered decklists and both pilots' results at strictly earlier events. For competitive players and deck-builders.

Horizon. One match. The model is given only what is known before the first card is played — the two registered 50-card lists and both pilots' results at strictly earlier events — and predicts that match's outcome. It is served on demand through a web UI (a player picks two decks and gets a probability), and the same model is scored nightly against every match that resolved since the previous run. The provider lists a tournament only once it has finished, so there is no in-progress feed to predict against in real time; the honest framing is on-demand serving plus nightly scoring, not live in-tournament inference. The reported winner is the label and is not computable from any feature.

Scope. Online Swiss rounds, predominantly best-of-one, at 32+ entrant events on play.limitlesstcg.com; top cut, byes, ties and unresolvable decklists excluded — 28,107 matches as of 2026-09-18, 82.1% of them from one recurring series, so claims cover online play only.

Success criterion. Baseline: a coin flip — Brier 0.2500, log loss 0.6931 — because the label is balanced by construction: seat 1 wins 50.5% of 29,004 decided pairings in the cached corpus (Wilson 95% CI 50.0–51.1%), leaving no majority-class rule and no rare positive class. Success = pooled over ≥ 6 held-out 28-day windows **split by event date, never by row** ($\geq 8,000$ matches): Brier ≤ 0.2490 with the skill interval (0.2500 – Brier, whole-event bootstrap) excluding zero, and calibration gaps ≤ 3 points in every 5-point favourite bucket with $n \geq 250$. Reproducing the existing archetype model out-of-time gives 0.2465; 0.2490 is the bar every six-window stretch in 14 months clears.

2 Originality & motivation

[Kiana writes this one — “why this problem, and why you” is the graded question and it has to be yours. Raw material: you already built and run the data pipeline this draws on; the modelling question is separating deck strength from pilot skill, which is a real confounder rather than a leaderboard; and competitive TCG match forecasting appears on neither the mlops-lab.ch showcase nor the KTH ID2223 lists. Check both explicitly and say so.]

3 Data source & features

Matches come from Limitless (play.limitlesstcg.com) via its public JSON API: `/api/tournaments,{id}/standings,{id}/pairings` — no auth, no scraping. GitHub Actions ingests daily at 06:07 UTC; finished events are immutable and fetched once. At 2026-09-18: 28,107 decided Swiss matches (both decklists resolved) over 193 events, 11,890 entries, 11,886 with full 50-card lists, growing ≈ 570 matches and 3.5 events weekly. One online series supplies 82%, so the population is online best-of-one.

Label. `pairings[].winner`, the username the platform records from the reported result; no decklist rule yields it. Seat 1 wins 50.5% (95% CI 49.9–51.1%, $n = 27,506$), so no rare positive class exists. Scarcity is in the archetype tail: 13.0% of held-out deck-sides pilot a leader unseen in training. They stay in — leader attributes carry them, and priors shrink to 0.5 at zero history.

Features, per side and differenced: `leader_id`, `leader_life`, `leader_power`, `leader_color_count`; deck aggregates `counter_2k_copies`, `avg_cost`, `high_curve_copies`, `big_body_copies`, `event_card_copies`, `distinct_cards`; and `player_prior_winrate` and `archetype_prior_winrate` over strictly earlier events.

Leakage. `placing`, `record` and `drop` are outcomes of the predicted event and are dropped at ingest: `placing` is null on 43.0% of 12,166 raw entries, all of which carry `drop`, so filtering `placing IS NOT NULL` would condition on finishing. Splits are rolling-origin by event date, never by row (one event's matches share decks and pilots): train up to T, test (T, T+28d]. The ingest must also persist the event date, which only the tournament index holds.

4 System design

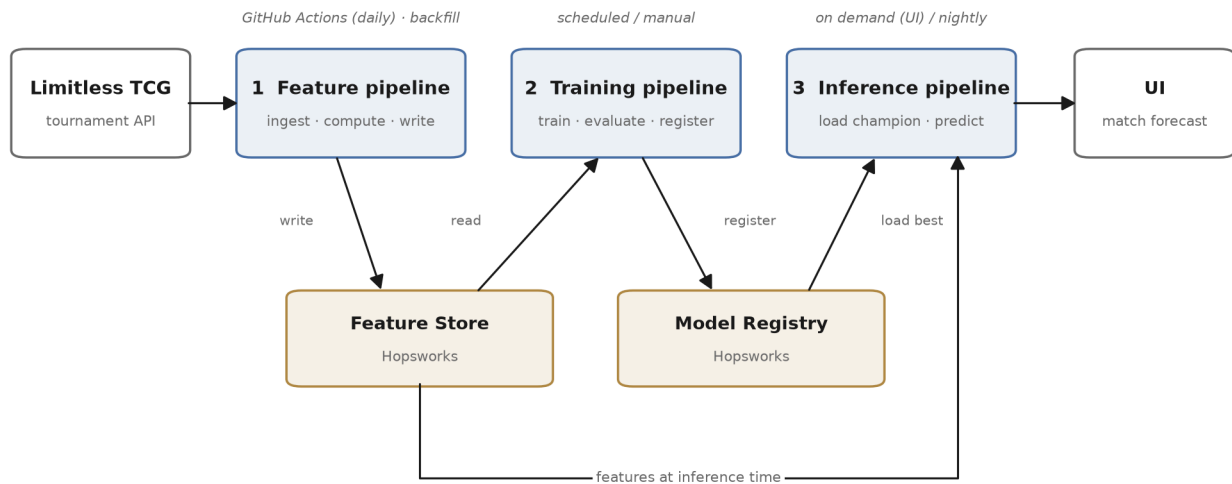


Figure 1: The three FTI pipelines. They are decoupled — none calls another; they meet only at the feature store and the model registry.

Core (ships first). A GitHub Actions job runs daily at 06:07 UTC, matching the source's ≈ 3.5 events a week. It pulls the Limitless API, drops `placing`, `record` and `drop` physically at the ingest boundary, persists each event's date from the tournament index — the pairings payload carries none, so without it no out-of-time split is possible — lands raw JSON in GCS and writes features to Hopsworks. Training reads the feature view, splits by event date, evaluates out of time, and registers the winner in the Hopsworks model registry under the alias `champion`. Cloud Run loads whatever currently carries that alias and serves win probabilities to the UI, logging each prediction so it can be scored once the match resolves; serving never calls the provider.

Stack. *Hopsworks*: point-in-time joins, so player history cannot include the event being predicted, and one hosted service covers both the feature store and the registry — promotion moves an alias rather than redeploying, and there is no tracking server to operate. *GitHub Actions*: schedules beside code and CI, and sole holder of the provider client — throttled to the published 50 requests per 5 minutes, finished events cached write-once. *Cloud Run*: container deploy, scales to zero between events. *GCS*: immutable landing zone, so features rebuild without re-fetching.

Optional stretch, outside the core: decklist drift monitoring, a second feed to dilute the $\approx 82\%$ -of-matches single-organiser concentration, and a calibration dashboard.

The repository is public: github.com/kianakiser/optcg-match-forecast.